

NOCR: End-to-End Structured Data Extraction from Historical Documents Using VLMs

Auke Rijpma
Utrecht University
a.rijpma@uu.nl

Rick Schouten
Utrecht University / National Library of the Netherlands

June 5, 2026

Abstract

Extracting structured data from historical archives is a time-consuming bottleneck in economic history research. Traditional approaches require optical character recognition (OCR) followed by manual postprocessing to structure the output. This two-step process is both inefficient and error-prone. This paper demonstrates that vision language models (VLMs) can collapse this pipeline into a single step, directly converting scanned images of historical documents into structured JSON output. This drastically reduces the need for post-processing. Using Google’s Gemini Flash models on handwritten Dutch tax records from 1899, we evaluate performance across multiple model versions (2.0, 2.5, and 3.0) and prompting strategies (zero-shot and few-shot). Our results suggest that this VLM-based extraction, which we call NOCR, is already viable for large-scale historical data projects, with clear evidence of rapid model improvement over time.¹

¹We thank William Skoglund, Jakob Molinder, and Ruben Peeters for useful comments. All code to run and explore the experiments is available at <https://github.com/HIP-NL/nocr-experiments.git>.

1. Introduction

In economic history there is heavy use of quantitative methods to analyze archival records, from tax assessments and census data to account books, probate inventories, and notarial records. The digitization of these sources has for a long time been a manual process, but is increasingly being automated. A common pipeline involves scanning physical documents, then processing them through optical character recognition (OCR) or handwritten text recognition (HTR) systems, and, finally, postprocessing makes the raw text output suitable for analysis.

This two-step approach has significant limitations. Traditional OCR/HTR systems already excel at transcribing text (Sang 2024; Leifert et al. 2024), but produce unstructured output that requires extensive manual intervention. As a result, the postprocessing step can consume more time than the initial transcription working with tabular or otherwise structured data. Researchers must write custom parsers, handle implicit information (such as repeated values in table columns), and correct systematic errors in the OCR output. This process and the final step in particular is, frankly, something which makes us miserable.

The main alternative described in the literature is another two-step approach, where first a segmentation model is trained to divide the scans in sections that map to the fields that the researchers want to extract. Next, standard OCR tools are applied to these segments. This approach has been used, for example, by Dahl et al. (2023) to transcribe nurse journals or by Blomqvist et al. (2022) to digitise a Swedish wealth tax of 1571 Bailey et al. (2023). A toolkit for this approach is described in Shen et al. (2021). While most authors report good performance, setting up and maintaining a two-step approach is potentially more complex compared to a one model approach as suggested here.

Recent advances in vision language models (VLMs) offer an alternative by directly converting scanned images into structured data. Rather than extracting text and then inferring structure, VLMs can be prompted to understand both the visual layout and semantic content of a document simultaneously. We call this approach “NOCR”, because we see it as a significant departure from traditional OCR methods. This approach even allows VLMs to handle complex cases that traditional OCR pipelines struggle with. Examples of this include splitting cell content in multiple fields, dealing with cells that implicitly copy values from rows above, or handling annotations written in the margins. VLMs furthermore have the capability to handle printed, typewritten, and handwritten text in many languages.

A further advantage is simplicity. A single API call replaces a multi-stage pipeline. Instead of maintaining separate OCR and postprocessing code, researchers can describe their desired output format in natural language and receive structured JSON in response, which can easily be converted to tabular data for downstream statistical analysis. This approach is particularly well-suited to historical documents, where the visual context often matters as much as the raw text.

In addition, VLMs also support in-context learning, where inputs and the expected

output are given to the models of examples. This approach can significantly improve the accuracy of the output on the final prediction (Agarwal et al. 2024). We show below that this is a cost-effective strategy when using commercial model providers, as input tokens are usually priced much lower than output tokens. In this way, we have been able to use this approach to extract the tax data from archival scans in the HIP-NL project (Rijpma et al. 2025).², handling 10000s records at very low cost, including typewritten, printed and handwritten records.

Related work includes Kim et al. (2025), who test a number of VLMs on early twentieth century probates as an alternative to traditional OCR. While they also find superior performance of VLMs in terms of lower CERs, they do not assess VLM’s abilities to extract structured data directly, rather focusing on line-by-line transcription. Griesshaber and Streb (2025) also VLMs to create datasets from late nineteenth and early twentieth-century German patent records. While they use VLMs throughout their pipeline, the text and feature extraction are separate steps. Moreover, we focus on the harder case of handwritten records, compared to the printed text used in the patents. In addition, we strongly emphasise the value of in-context learning as a strategy to increase accuracy while keeping cost under control, a point only made in passing by Kim et al. (2025). Ribeiro et al. (2026) finetune an early local VLM, DONUT, which we see as an important alternative for larger-scale cases. However, finetuning will not be a cost-effective strategy for all types of data. Probably, many researchers working with scanned text sources have already discovered similar approaches independently or through the grapevine.

In this study we demonstrate the viability of VLM-based extraction using Google’s Gemini Flash family of models (Gemini Team et al. 2023) applied to handwritten Dutch tax records from 1899. Our evaluation addresses three questions. First, how well do VLMs perform at combined OCR and structuring tasks on historical documents (accuracy)? Second, what are the cost-performance tradeoffs across different models and prompting strategies? Third, are these models improving, and what does this suggest for future applications?

Six model variants are tested (ranging from lightweight “flash-lite” models to the latest “flash-3” release), two prompting strategies (zero-shot and few-shot with examples), and extended reasoning modes. Experiments are done on a dataset containing 40 individual tax records with ground truth annotations. In addition, the favoured model is applied to the full 1899 tax records, containing over 10,000 households. The results suggest that VLM-based extraction is not only viable but often superior to traditional approaches, particularly when total workflow time is considered.

²<https://www.hipnl.nl/>

2. Data and Methods

2.1 The Utrecht 1899 Tax Records

Our test dataset comes from the municipal archives of Utrecht in the Netherlands (Het Utrechts Archief). The records document household income tax assessments for the year 1899, with each entry recording the household head’s initials and surname, address, tax bracket, and taxes due. The appendix provides an example scan and record.

While these records are fairly clear and consistent as far as historical archival material goes, the documents present several challenges typical of historical economic data. First, they are handwritten and have corrections in the cell in a second hand. Names and addresses use 19th-century Dutch spelling conventions and abbreviations. Titles like “Wed.” (widow), “Mej.” (miss), and “geb.” (born, indicating maiden name) must be correctly interpreted and extracted in a standardised way. Moreover, multiple variables are combined in a single cell, such as title, initials, surname, and maiden name, which we would want to split. The records use a five-column layout where street names are often omitted when they match the previous row. A correct transcription requires understanding this implicit repetition. Finally, the numeric tax columns use different conventions from the other columns. For streets, empty cells indicate repetition, while an empty final column means “oo” cents.

Our ground truth consists of four scanned pages (two double-page spreads, split into left and right images) containing 40 individual records, with data split over five columns. Each record includes nine fields that we want to extract: serial number (volgnummer), title, initials, surname, maiden name, street, house number, tax class, and tax amount. Note that these are to be extracted directly, with little to no postprocessing. Ground truth transcriptions were created by manual review. While the total number of scans is low, there are enough cells and characters to calculate meaningful performance metrics, while keeping the cost of running multiple experiments to a minimum.

2.2 Experimental Design

model name	short description
gemini-2.0-flash-lite	Lightweight, lowest cost
gemini-2.0-flash	Standard 2.0 version
gemini-2.5-flash-lite	Lightweight 2.5 generation
gemini-2.5-flash	Standard 2.5 version
gemini-3-flash	Latest 3.0 generation
gemini-3.1-flash-lite-preview	Lightweight 3.1 generation

We evaluated five Gemini Flash models. For each model, we tested multiple configurations. For prompting, we tested a zero-shot (prompt and target image only)

versus few-shot (three example images with ground truth JSON, followed by target image) strategy. Few-shot or in-context learning allows the model to learn formatting conventions and error patterns from examples, at the cost of longer context windows and higher input token usage.³

We also tested for the impact of standard inference versus extended thinking (2000-token budget). The thinking mode allows models to generate internal reasoning chains before producing output, potentially improving accuracy on complex tasks. This comes at the cost of higher output token usage.

Our prompt provides detailed instructions for each field, including specific guidance on edge cases (e.g., “If a street is omitted because it is the same as the previous row fill in the missing value with the last explicitly stated one”). The prompt was iteratively built to try and catch obvious errors in the output. The full prompt appears in Appendix B.

We use a leave-one-out evaluation design: for few-shot experiments with four total images, we use three as examples and the fourth as the target, rotating through all combinations. This ensures each image is evaluated exactly once per model-strategy combination. The Gemini API is stateless, so there is no contamination of the evaluation data.

2.3 Evaluation Metrics

We measure the approaches at three levels:

Character Error Rate (CER). For each record, we calculate the Optimal String Alignment distance (character-level edit distance, including transpositions) between predicted and ground truth values across all nine fields (Loo 2014), divided by the total number of characters in the ground truth. This gives a fine-grained measure of transcription accuracy.

Cell Error Rate. The share of fields (out of nine per record) that contain any error. A field with a single incorrect character counts the same as a completely wrong field. This metric better reflects the downstream cost of errors, since even small mistakes can require manual correction. It should however be noted that some cell errors are not as consequential as others. For example, a one-character mistake in a first name is arguably not that important in a situation where spelling variation between names was not uncommon to begin with.

Cost. We calculate the total cost per page based on Gemini API pricing (as of January 2025), accounting for input tokens, output tokens, and thinking tokens where applicable. Prices range from \$0.075 per million input tokens (flash-lite) to \$3.00 per million output tokens (flash-3).⁴

All 40 records are evaluated for each model-strategy combination, giving 800 total scan-level predictions and 7200 predicted cells across our experimental design.

³Niels Rogge for the idea to do this: <https://x.com/NielsRogge/status/1901373186043998525>.

⁴<https://ai.google.dev/gemini-api/docs/pricing>. Unfortunately, the generous free tier of the Gemini API (200 requests, one million tokens per day) is no longer available.

3. Model Performance

3.1 Overall Accuracy

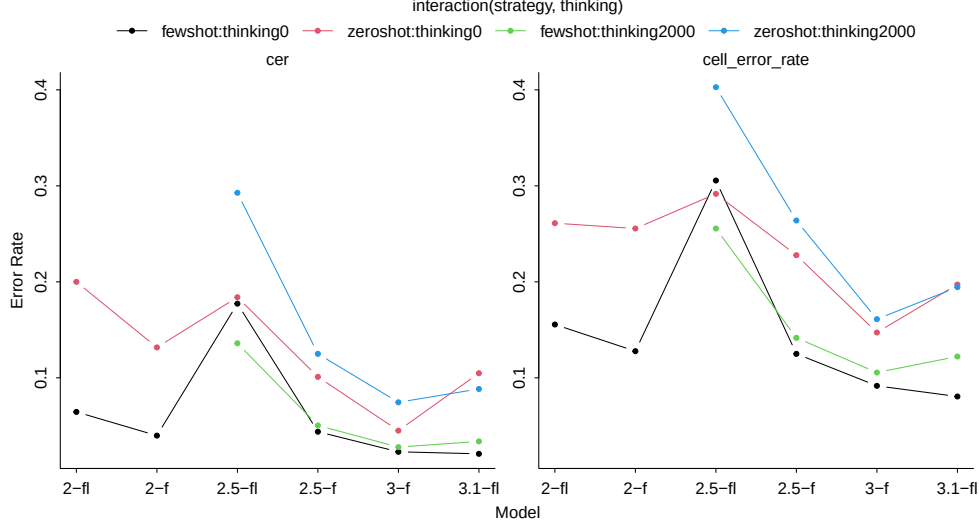


Figure 1: Error rates by model and strategy

Figure 1 shows character and cell error rates across all model-strategy combinations. The first thing to observe is that the trend in model performance improves over the generations. Even if some models deteriorate over previous generations in some strategies, overall we should expect newer models to perform better.

Few-shot learning is shown to provide substantial benefits. Across all models, few-shot prompting strongly improves error rates. For 3.0-flash, few-shot reduces CER from 4.5% (zero-shot) to 2.3% (few-shot). The effect is even stronger for earlier model versions—Gemini 2.0’s CER drops from 13.2% to 4.0% with few-shot examples. The best error rate, 2.1% is obtained using few-shot prompting for the 3.1-flash-lite model. This suggests that showing the model correct formatting is very valuable. Interestingly, zero-shot Gemini 3.0 outperforms few-shot Gemini 2.0 and 2.5, suggesting that model quality can substitute for in-context learning to some degree. However, the combination of best model and few-shot prompting remains superior overall.

Extended thinking shows poor results. The 2000-token thinking budget produces inconsistent effects. For all models except 2.5-Flash-lite, thinking slightly degrades performance. Notably, 3.0-Flash often refuses to use extended thinking even when prompted, reverting to standard inference. This suggests that VLMs do not need explicit chain-of-thought to extract data from relatively straightforward and consistent visual inputs.

Lite models show varying performance, with standard models often outperforming their “lite” counterparts. This suggests that model compression strategies such as distillation and quantisation can hurt performance. As the lite models are also

Table 2: Top 5 models and strategies by error rate.

model	strategy	thinking	cer	cell_error_rate
gemini-3.1-flash-lite-preview	fewshot	thinkingo	0.021	0.081
gemini-3-flash-preview	fewshot	thinkingo	0.023	0.092
gemini-3-flash-preview	fewshot	thinking2000	0.028	0.106
gemini-3.1-flash-lite-preview	fewshot	thinking2000	0.034	0.122
gemini-2.0-flash	fewshot	thinkingo	0.040	0.128

the fastest and the cheapest, this has important implications for large-scale and cost-sensitive applications. At the same time, it should be noted that the 3.1-Flash-Lite model is the best performing one in combination with a few-shot prompting strategy.

3.2 Best Performing Configurations

The top five configurations ordered by character error rate are listed in 2. Looking at the levels, even the best-performing model still makes errors on roughly one in twelve fields. Manual inspection reveals these are often challenging cases: ambiguous handwriting (particularly ‘o’ vs ‘8’ and similar letters), unusual abbreviations, or edge cases in the table structure. The best performance, a 2.1% character error rate, translates to roughly one in every 50 character wrong. These numbers are in line with, and often improve on traditional OCR methods for historical handwriting⁵ While this is high enough that output requires review, in our experience the process is vastly more efficient than manual transcription from scratch.

3.3 Error Patterns

Examining specific errors reveals predictable patterns. Common mistakes include confusing ‘o’ and ‘8’ in sequential numbers and house numbers (e.g., predicting 1061 vs ground truth 1861), incorrectly handling of implicit street name repetition, and errors in parsing decimals separated by a vertical bar in tax amounts.

These error types suggest potential improvements through more detailed prompt engineering, fine-tuning on similar documents, or automated error catching. The fact that errors cluster around specific challenging cases (rather than being randomly distributed) is encouraging. With domain-specific training, even higher accuracy is achievable.

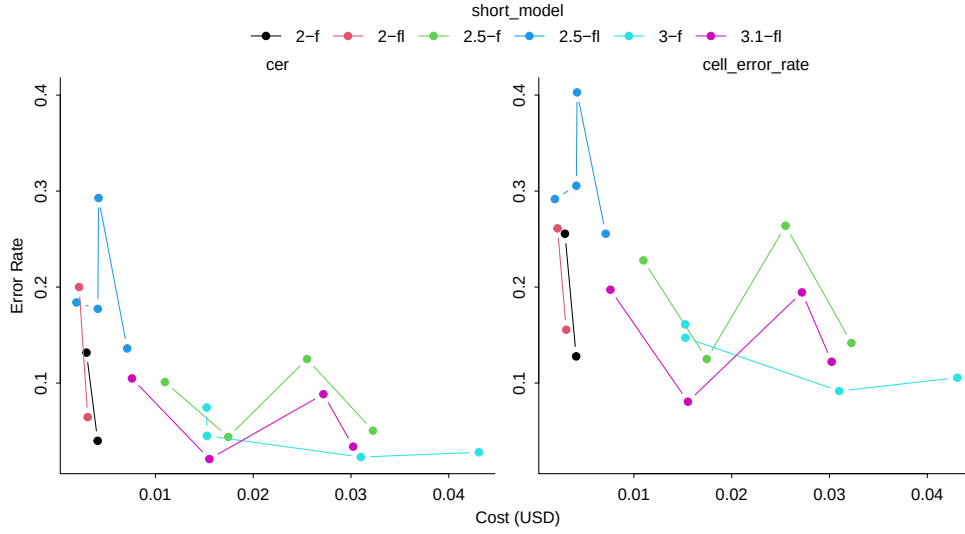


Figure 2: Error rates by cost and model

4. Cost-Performance Analysis

4.1 The Cost-Accuracy Frontier

Figure 2 plots error rates against total cost per page for all configurations and table 3 lists the best-performing models by cost. Cost varies dramatically across configurations, from approximately \$0.002 per page (Gemini 2.0 Flash Lite zero-shot) to \$0.043 per page (3.0-Flash few-shot with thinking). This 20-fold range can accommodate different use cases. The cheapest models combined with a few-shot strategy (2.0 Flash or 2.0 Flash Lite) can achieve strong performance (4.4 and 6.5% CER) for low prices (\$0.004 and \$0.003 per page respectively). Better performance can be obtained with a higher budget. The 3.0-Flash models provide some of the best performance, but at the highest prices (c. 2.5% at \$0.03–0.04 per scan). We think the optimal model in these experiments is the combination of a 3.1-flash-lite model with few-shot prompting, providing the highest performance at a mid-range budget (0.016). The cheapest models here could process a thousand scans pages for about \$3; the mid-range option would cost \$15, a fraction of manual transcription costs, with accuracy already sufficient for most quantitative analyses. To put those pages counts into perspective, the full 1899 tax records for Utrecht adds up to 1108 scans; by 1920, that number had grown to 2828.

4.2 Scaling Considerations

Several factors could further reduce costs at scale. First, context caching could reduce costs. If the few-shot examples remain constant across pages from the same document type. Gemini’s context caching feature could reduce input token costs

⁵E.g. 3.7% on typewritten text by Amujala et al. (2022); 8.4% on 16th century handwriting by Blomqvist et al. (2022); 5.5% on nineteenth-century handwriting by Sang (2024); 5% by Muehlberger et al. (2019); 3% by Koert et al. (2024).

Table 3: Model cost and performance for models with with at least ten percent CER, ordered by costs

cost	model	strategy	thinking	error_rate
0.003	gemini-2.0-flash-lite	fewshot	thinkingo	0.065
0.004	gemini-2.0-flash	fewshot	thinkingo	0.040
0.015	gemini-3-flash-preview	zeroshot	thinking2000	0.075
0.015	gemini-3-flash-preview	zeroshot	thinkingo	0.045
0.016	gemini-3.1-flash-lite-preview	fewshot	thinkingo	0.021
0.017	gemini-2.5-flash	fewshot	thinkingo	0.044
0.027	gemini-3.1-flash-lite-preview	zeroshot	thinking2000	0.088
0.030	gemini-3.1-flash-lite-preview	fewshot	thinking2000	0.034
0.031	gemini-3-flash-preview	fewshot	thinkingo	0.023
0.032	gemini-2.5-flash	fewshot	thinking2000	0.050
0.043	gemini-3-flash-preview	fewshot	thinking2000	0.028

by up to 75% for few-shot scenarios, making the premium tier more affordable.⁶ Because we varied the examples per prompt in these experiments, we could not use this feature here.

Likewise, processing documents in batches could enable further optimizations and volume discounts. This reduces costs by 50%. It is straightforward to set up, and the only downside is a delay of up to 24 hours.

Mixed strategies could also be valuable. Projects could use expensive models for challenging documents and cheaper models for routine cases, optimizing the overall cost-accuracy tradeoff. This would require a reliable method to assess page difficulty.

Finally, it is worth discussing VLM pricing. While there have been substantial reductions, e.g. the costs of 2.5 at its introduction was a factor ten lower than 2.0 at its introduction. Moreover, the overall trend in LLM pricing is negative, decreasing by c. 25% each year. At the same time, if we omit the introduction pricing of 2.5 compared to 2.0, each Gemini Flash model was more expensive than the one that came before.⁷ Moreover, the generous free tier of the Gemini API has been removed.

5 Performance on the full 1899 Utrecht tax data

To assess the viability of the NOCR workflow at scale, we report on our application of the pipeline to a complete corpus of scans from the 1899 Utrecht tax records, the full version of the data we used for the experiments above. In addition to showing the viability, it can also highlight some practical lessons. This dataset consists of 10659 households spread over 1074 scans. This task was done in April 2025, using

⁶<https://ai.google.dev/gemini-api/docs/caching?lang=python>.

⁷<https://github.com/sanando/llmpricing>. No scholarly review of LLM pricing trends could be found at the time of writing.

Gemini 2.0 Flash. The outputs generated by the VLM were subsequently checked and corrected by research assistants before adding them to the HIPNL database.⁸ The resulting manually verified tables can serve as ground truth data, allowing us to evaluate the performance of our few-shot prompting strategy on a full production run.

To quantify the performance of our approach, we evaluated the data on a by-column basis using string similarity scores derived from Levenshtein distance, specifically utilizing the `stringsim` function from the R `stringdist` package (Loo 2014). The Levenshtein distance counts the total number of deletions, insertions, and substitutions needed to change the original model response in the corrected field.

For most fields, such as surnames, titles, tax classes, and tax amounts, the mean similarity scores between the VLM’s raw output and the RAs’ corrected ground truth are around 0.95 or higher, and cell error rates are generally low, indicating highly accurate transcription (table 4). Performance on the initials field is somewhat lower, indicating that the model, not unlike an inexperienced human transcriber, struggles somewhat with capitals without the context of the rest of the word.

Table 4: Mean Similarity Scores and Cell Error Rates per Column

Column	Mean_Similarity	Cell_Error_Rate
class	0.954	0.053
house_number	0.927	0.111
initials	0.928	0.175
street	0.788	0.465
surname	0.944	0.211
tax	0.948	0.122
title	0.982	0.021
volgnummer	0.929	0.202

Overall, the performance of the approach in a project like this is highly encouraging. We achieved a mean Character Error Rate (CER) of 12.22%, alongside an overall cell error rate of 17%. As will be discussed below, the higher CER compared to the model’s performance above is due to the handling of the street column. Omitting this column results in a CER of 4.2%, in line with the results of the experiments above.

However, the column-level evaluation also highlighted one specific area of vulnerability: the `street` column, which exhibits a noticeably lower mean similarity score of 0.788. The histograms below illustrate this disparity, showing a strong concentration of perfect or near-perfect scores for `surname` but a much wider spread of errors for `street`.

⁸In addition, the NOCR approach was also used for all printed and typewritten records. As these are far easier for the model, performance was superior to the results presented here.

Table 5: The 10 most frequent mistakes in the automated transcription of the 1899 Utrecht tax records

Field	Original	Corrected	Count
Tax	0.25	8.25	335
Tax	10	18	285
Class	1	7	69
Initials	A.	M.	58
Initials	A.	H.	46
Class	21	2	41
Class	1	3	36
Class	1	2	36
Tax	80.5	88.5	32
Tax	100	180	31

Perhaps the most important consideration is that the raw JSON responses returned by the model could be converted into structured, tabular data “as is.” We did not need to implement any rule or model based post-processing pipelines to handle formatting. RAs could go straight to work in the tables, where potential mistakes could be easily flagged (e.g. and discrepancies between the tax due and tax bracket columns). The required manual corrections were generally minimal, the most important ones being correcting 0-8 mistakes (table 5)

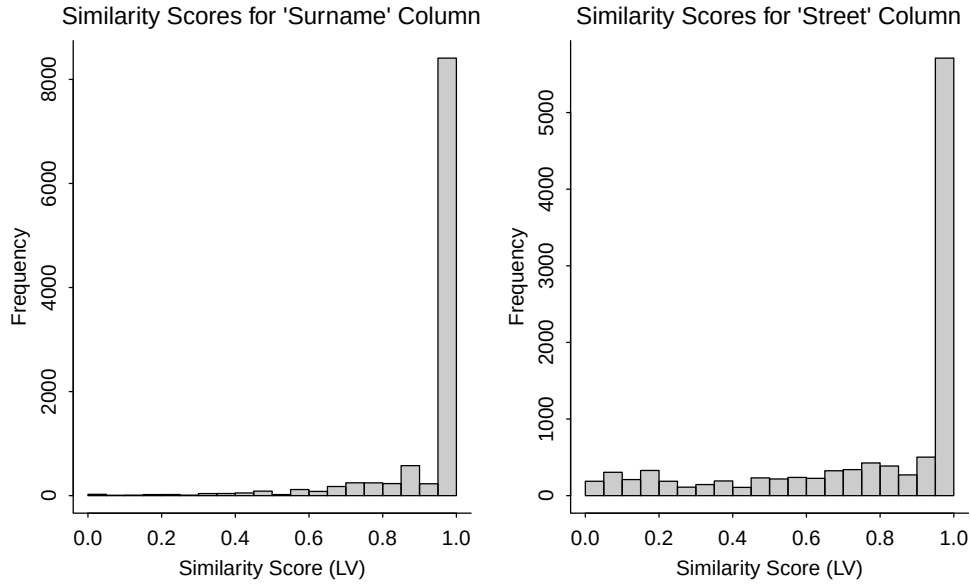


Figure 3: Distribution of row-wise similarity scores between the surname and street columns.

This poor performance in the street column is largely due to the structural logic of the documents causing significant cascading errors. In the original 1899 ledgers, clerks left the street column empty for consecutive households living on the same

street. To create a complete dataset, we instructed the model to carry these street observations forward when encountering empty spaces. As a result, mistakes in this column – pushing a maiden name into the address column or even an technically correct one such as filling in a new address that was added in pencil – this error would be pushed down into multiple empty cells (see appendix). In cases like this, a single extraction error penalizes the model across multiple rows, artificially inflating the error rate for the street column relative to the actual transcription capability of the model.

To isolate these error types, we used the “trafos” option in the `adist()` function in R to break down the string distance in the individual operations needed to optimally change one string in another. We use this to find series of at least four deletions and insertions. This allows us to separate structural mistakes like omitting words from the correct column or inserting words from another field from character-level mistakes. We identify these structural mistakes by searching for long, consecutive blocks of deletions or insertions in the transformation strings.

Table 6: Distribution of Error Types (Structural Transformations) per Column

Column	None	Transcription	Omission	Inclusion	Omission and inclusion
class	10089	565	0	0	0
house_number	9476	1119	30	29	0
initials	8791	1840	6	17	0
street	5698	4258	349	344	5
surname	8404	2072	137	41	0
tax	9352	1301	0	1	0
title	10427	189	31	7	0
volgnummer	8504	2150	0	0	0

Table 6 shows that such structural mistakes mostly occur in the street column. The fill-down prompt dynamic creates nearly 700 combined structural errors in `street`, compared to very few in other textual or numerical columns. For numeric columns like `tax` and `class`, structural mistakes are practically nonexistent, remaining limited to small transcription errors.

6. Discussion

This paper demonstrates that VLM-based extraction of structured historical data is not just viable but competitive with traditional approaches, and can even outperform it. The most important finding is that direct scan-to-JSON works reliably. Even complex tabular layouts with implicit data and historical handwriting can be accurately *and directly* parsed into structured output without separate OCR and postprocessing steps. Applying this to large corpuses of sources results in databases that are easily reconstructed and checked. Error rates overall are workable, though care should be taken in dealing with the logical structure of the scans. In our Utrecht 1899 case, most of the error rate could be traced to the

decision to let the model take care of the last-observation-carried-forward logic in the scans, which resulted in carrying forward mistakes as well.

A further conclusion is that few-shot learning is a highly effective way to improve performance. The dramatic improvement from showing just three examples suggests that in-context learning remains valuable even as base model capabilities improve. Moreover, this approach makes relatively cheaper models interesting options.

Cost-performance tradeoffs are also favorable. For \$0.004 per page, we achieve error rates low enough to support quantitative analysis with modest manual review. This is orders of magnitude cheaper than manual transcription (estimated at \$1–5 per page for professional transcribers). Few-shot prompting also provides some of the most interesting price-performance points as the input tokens for the extended prompt are usually priced far lower than output tokens.

Our results show that performance is improving rapidly. The 65% reduction in error rates from Gemini-Flash 2.0 to Gemini-Flash 3.0 (released less than a year apart) suggests we are on a steep improvement curve. Gemini-3.1-flash-lite-preview, released in March 2026, presented another improvement compared to 3.0. The flipside to this is that models are deprecated rapidly as well. At the time of writing, the Gemini 1.5 models that were originally part of our tests and setup for printed documents are no longer available through the API.

There are more caveats to our findings. We evaluate only one document type (Utrecht tax records from one year). Generalization to other formats, languages, or time periods requires further testing. Anecdotally, we have found this approach to work well on most printed, typewritten, and handwritten records from the eighteenth century onwards. A key constraint is that we have found that applying these methods to pages with tables consisting of many rows without gridlines can result in output where the columns are out of sync which can take a lot of time to fix.

Open source models are a promising avenue for further research. Many open VLMs are available that can be run locally (e.g. Qwen2-VL, Wang et al. 2024; PaliGemma Beyer et al. 2024). While often less capable than commercial models offered over APIs, fine-tuning on historical documents could close the gap while eliminating API costs and data privacy concerns. It would also avoid vendor lock-in, model deprecation, or pricing changes that could disrupt projects. Fine-tuning smaller models on large corpora of historical documents (perhaps using cheaper models to generate noisy labels for expensive models to refine) could substantially improve accuracy on domain-specific challenges like historical handwriting and orthography. Our own experiments in finetuning the DONUT family of models were promising in this regard (Kim et al. 2022; see the follow-up in Ribeiro et al. 2026), but using commercial models with in-context learning through an API was more efficient to set up for our use case of 100–1000s of scans per tax record type. Using few-shot learning on local models generally did not give satisfying results, and full finetuning of models seems necessary, increasing the cost.

Getting models to provide confidence scores for each extracted field would also be valuable. This would enable a hybrid workflows where high-confidence extractions are accepted automatically while uncertain cases receive manual review, or follow-up with a more powerful and expensive model. A simple but cost-inefficient way to do this would be to have multiple model runs and have them vote on the output.

Finally, the tabular tax records used represent one type of document in the vast space of historical records available in the archives. Applying similar methods to census records, notarial archives, merchant ledgers, and correspondence would demonstrate broader applicability.

The viability of VLM-based extraction has significant implications for historical research. As costs fall and accuracy improves, creating structured datasets from archival sources becomes feasible for individual researchers with limited funding. Projects involving millions of pages become tractable. The entire Utrecht tax series (hundreds of volumes) could be digitized for a few thousand dollars rather than hundreds of thousands. The key constrain, rather, is whether archives can provide their records as high-quality scans.

Appendix A: Sample Scans and Ground Truth

The scans containing the tax records look as follows:

Below is a sample ground truth record demonstrating the structure and complexity of the data:

```
{
  "volgnummer": 1861,
  "title": null,
  "initials": "G.",
  "surname": "Fukkink",
  "maiden_name": null,
  "street": "Oudekamp",
  "house_number": "10",
  "class": 2,
  "tax": 4.5
}
```

This record shows a typical entry: sequential number 1861, no title, initials “G.”, surname “Fukkink” (note the challenging double-k and -ink ending), address at Oudekamp 10, tax class 2, and tax amount of 4.5 guilders (using decimal notation after conversion from the vertical-line separator in the original).

More complex cases include widows with maiden names:

```
{
  "volgnummer": 1870,
  "title": "Wed.",
  "initials": "H.Th.E.",
  "surname": "Kuijper",
  "maiden_name": "J.M. Leuven",
  "street": "Oudkerkhof",
  "house_number": "5",
  "class": 1,
  "tax": 1.5
}
```

Here “Wed.” (Weduwe, widow) as the title, multi-part initials “H.Th.E.”, and the maiden name “J.M. Leuven” (indicated by “geb.” in the original scan) demonstrate the richer structure that VLMs must correctly parse.

187

W I J K

Volgnummer.	NAMEN VAN DE BELASTINGSSCHULDIGEN.	WOONPLAATS. Straat, gracht, enz. en nummer.	Klasse.	Bedrag van den aanslag à 3 pct.		N ^o . van liet
11061.	G. Fuukink	Oudekamp N ^o . 10	2	1	4 50	14, 59 111
11062.	C.B. Beijerman	N ^o . 12	10		60	58 113
11063.	J. van Rijveld	N ^o . 16	2		4 50	15
11064.	J.C.E. Prockie	N ^o . 16	1		1 50	74
11065.	G. van Baak	N ^o . 10	9		49 50	8
11066.	J.H. Knobbeut	Oudekerkhof N ^o . 3	1		1 50	65
11067.	J. van der Pijl	N ^o . 3 bis	2		4 50	112
11068.	O. J. Verkerk	N ^o . 5	5		10	50 116
11069.	brekend bevrind A.W. Hoffferich bloedchaan w.g. kerkhof w. 12 bis	N ^o . 5	1		1 50	114
11070.	Wid. H. Th. P. Kuiper geb. J. M. Leunon Oudekerkhof w. 156 bis	N ^o . 5	1		1 50	116
Totaal der bladzijde.				1	147	

529. Model No. 8. I. B.

Figure 4: Handwritten municipal income tax records from the Utrecht Archives, 1007-2 Gemeentebestuur van Utrecht 1813-1969, no 6613

Appendix B: Prompt

The full prompt used for all experiments:

Perform the following task using step-by-step reasoning.

Extract the data from this scan and return it as json.

The scan contains five columns describing individuals being taxed in the Dutch city of Utrecht in 1899. Extract the following information and return it as json:

- volgnummer, the sequential number of the individual in the scan, an integer number at most four digits long. If there are more digits, the first one is probably from the previous page and should be ignored.
- title, a title such as 'Wed.', 'Dr.', 'Mej', 'jKHR', 'jkvr', 'geb', 'vert', 'p/a', 'cur'. If the scan does not contain a title, leave this empty (null).
- initials, the initials of the individual.
- surname, the surname of the individual.
- maiden name, the full maiden name of the individual, below the husband's name and indicated with 'geb' (for 'geboren'), typically provided if she is a widow.
- street, the street in Utrecht of the individual, always in the 'woonplaats' column. If a street is omitted because it is the same as the previous row fill in the missing value with the last explicitly stated one. If the scan does not start with a street to fill in, leave the street empty. Below the name another address may be written if the individual moved. You should ignore this and only provide the original address from the woonplaats column.
- house_number, the house number of the individual, after the printed 'No.'. This is an integer number, sometimes followed by a letter (a, b), a superscript number, or a superscript text (usually 'bis'). Usually, the house numbers within a street will be increasing.
- class, the tax class of the individual, in the 'klasse' column, an integer number. This should always be transcribed. In the first row there will be an 'f' next to the class, which should be ignored.
- tax, the tax amount due for the individual, in the 'Bedrag van den

aanslag' column. This is a decimal number, where a vertical printed line is used as the decimal separator. A quote mark can also follow the vertical line instead of the usual two digits, which should be transcribed as two zeroes. A little bit of the next column might be visible, but this should be ignored, only the part in the 'Bedrag van den aanslag' column should be transcribed.

Carefully distinguish between the digits '0' and '8'. The digit '0' is a simple oval with a continuous outline, maybe open at the top if the writer was in a hurry, while '8' consists of two stacked loops, typically crossing the loop.

The prompt combines general instructions with specific edge case handling. Key features include explicit field descriptions, where each JSON field is described in detail with examples.⁹ Where a field can only contain a number of options, these are specified. There is edge case guidance to instruct the model how to handle implicit data (street repetition), moved addresses, and formatting conventions. Character-specific instructions for the distinction between a 'o' vs '8', though this was not very succesful. We explain column headers and layouts helps the model understand the visual organization. And finally, we request JSON to ensure structured, parsable output. The “step-by-step reasoning” prefix was included to encourage extended thinking, though as noted earlier, Gemini 3.0 often ignores this instruction when it considers reasoning unnecessary.

⁹In addition, it can also be useful to specify the datatype for each field.

Table 7: Example of carried forward, original json response and corrected data

street_orig	street_corr	street_sim
Lepelenburg	Lepelenburg	1.000
Lepelenburg	Lepelenburg	1.000
Lichthgaard	Lichtegaard	0.909
Lepelenburg	Lichtegaard	0.273
Lepelenburg	Lichtegaard	0.273
Lepelenburg	Lichtegaard	0.273
Lepelenburg	Lichtegaard	0.273
Lepelenburg	Lichtegaard	0.273
Lepelenburg	Lichtegaard	0.273
Lepelenburg	Lichtegaard	0.273

Appendix C: Example of fill-down issues

A F D E E L I N G						
Volgnummer.	N A M E N VAN DE BELASTINGSSCHULDIGEN.	W O O N P L A A T S.	K l a s s e.	Bedrag van den aanslag 3 à pCt.		A F S N ^o van het journal.
		Straat, gracht, enz en nummer.				
921.	P. A. Roepen Boeck	Lepelenburg N ^o . 2	15	151	50	140772
922.	P. J. van Rijn	No. 3 ^{de}	3	8	25	58982
923.	J. G. L. Lamaine	Lichtegeard N ^o . 11	2	4	50	157262
924.	A. G. van Deth	No. 2	4	12	75	153041
925.	J. Lertou	No. 3	11	73	50	32266
926.	M. C. A. Molenkamp	No. 4	3	8	25	109422
927.	Wid. W. J. Bijleveld geb. J. W. Lertou	No. 4 ^{de}	3	8	25	32255
928.	M. van der Wijden	No. 5	2	4	50	9952
929.	H. Rutgers	No. 6	9	49	50	1110 5026 9481 10587
930.	Wid. J. F. van der Linden geb. J. J. Wijnmalen	No. 6	1	1	50	2381
Totaal der bladzijde.				322	50	

Figure 5: Example of fill down dynamic in Utrecht 1899 HO tax. Source: HUA 1007-2 no 6612

References

- Agarwal, Rishabh, Avi Singh, Lei Zhang, et al. 2024. “Many-Shot in-Context Learning.” *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Red Hook, NY, USA), NIPS ’24.
- Amujala, Someswar, Angela Vossmeier, and Sanjiv R. Das. 2022. “Digitization and Data Frames for Card Index Records.” *Explorations in Economic History*, July 15, 101469. <https://doi.org/10.1016/j.eeh.2022.101469>.
- Bailey, Martha J., Susan H. Leonard, Joseph Price, Evan Roberts, Logan Spector, and Mengying Zhang. 2023. “Breathing New Life into Death Certificates: Extracting Handwritten Cause of Death in the LIFE-m Project.” *Explorations in Economic History* 87 (January): 101474. <https://doi.org/10.1016/j.eeh.2022.101474>.
- Beyer, Lucas, Andreas Steiner, André Susano Pinto, et al. 2024. *PaliGemma: A Versatile 3B VLM for Transfer*. arXiv:2407.07726. arXiv. <http://arxiv.org/abs/2407.07726>.
- Blomqvist, Christopher, Kerstin Enflo, Andreas Jakobsson, and Kalle Åström. 2022. “Reading the Ransom: Methodological Advancements in Extracting the Swedish Wealth Tax of 1571.” *Explorations in Economic History*, July 16, 101470. <https://doi.org/10.1016/j.eeh.2022.101470>.
- Dahl, Christian M., Torben S. D. Johansen, Emil N. Sørensen, Christian E. Westermann, and Simon Wittrock. 2023. “Applications of Machine Learning in Tabular Document Digitisation.” *Historical Methods: A Journal of Quantitative and Interdisciplinary History* 56 (1): 34–48. <https://doi.org/10.1080/01615440.2023.2164879>.
- Gemini Team, Rohan Anil, Sebastian Borgeaud, et al. 2023. *Gemini: A Family of Highly Capable Multimodal Models*. arXiv. <https://doi.org/10.48550/ARXIV.2312.11805>.
- Griesshaber, Niclas, and Jochen Streb. 2025. *Multimodal LLMs for Historical Dataset Construction from Archival Image Scans: German Patents (1877-1918)*. arXiv:2512.19675. arXiv. <https://doi.org/10.48550/arXiv.2512.19675>.
- Kim, Geewook, Teakgyu Hong, Moonbin Yim, et al. 2022. “OCR-Free Document Understanding Transformer.” In *Computer Vision – ECCV 2022*, edited by Shai Avidan, Gabriel Brostow, Moustapha Cissé, Giovanni Maria Farinella, and Tal Hassner, vol. 13688. Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-19815-1_29.
- Kim, Seorin, Julien Baudru, Wouter Ryckbosch, Hugues Bersini, and Vin-

- cent Ginis. 2025. *Early Evidence of How LLMs Outperform Traditional Systems on OCR/HTR Tasks for Historical Records*. arXiv:2501.11623. arXiv. <https://doi.org/10.48550/arXiv.2501.11623>.
- Koert, Rutger van, Stefan Klut, Tim Koornstra, Martijn Maas, and Luke Peters. 2024. “Loghi: An End-to-End Framework for Making Historical Documents Machine-Readable.” In *Document Analysis and Recognition – ICDAR 2024 Workshops*, edited by Harold Mouchère and Anna Zhu. Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-70645-5_6.
- Leifert, Gundram, C. A. Romein, Achim Rabus, Phillip Benjamin Ströbel, and Tobias Hodel. 2024. “Transkribus and Beyond: Pioneering the Future of Transcription Technology.” February 15.
- Loo, Mark van der. 2014. “The Stringdist Package for Approximate String Matching.” *The R Journal* 6 (1): 111–22. <http://CRAN.R-project.org/package=stringdist>.
- Muehlberger, Guenter, Louise Seaward, Melissa Terras, et al. 2019. “Transforming Scholarship in the Archives Through Handwritten Text Recognition: Transkribus as a Case Study.” *Journal of Documentation* 75 (5): 954–76. <https://doi.org/10.1108/JD-07-2018-0114>.
- Pedersen, Bjørn-Richard, Einar Holsbø, Trygve Andersen, et al. 2022. “Lessons Learned Developing and Using a Machine Learning Model to Automatically Transcribe 2.3 Million Handwritten Occupation Codes.” *Historical Life Course Studies* 12 (January): 1–17. <https://doi.org/10.51964/hlcs11331>.
- Ribeiro, Leonardo Costa, Jonatan Andersson, William Skoglund, Jakob Molinder, and Martin Önnersfors. 2026. *Automated Historical Census Digitization Using Image Augmentation and Transformer-Based Methods*. {EHES} Working Paper No. 298.
- Rijpmma, Auke, Eva van der Heijden, Alex Nagel, Paul Puschmann, and Rick Schouten. 2025. *An Historical Income Panel for the Netherlands, 1850–1920*.
- Sang, Erik Tjong Kim. 2024. *REE-HDSC: Recognizing Extracted Entities for the Historical Database Suriname Curacao*. arXiv:2401.02972. arXiv. <http://arxiv.org/abs/2401.02972>.
- Shen, Zejiang, Ruochen Zhang, Melissa Dell, Benjamin Charles Germain Lee, Jacob Carlson, and Weining Li. 2021. “LayoutParser: A Unified Toolkit for Deep Learning Based Document Image Analysis.” In *Document Analysis and Recognition – ICDAR 2021*, edited by Josep Lladós, Daniel Lopresti, and Seiichi Uchida, vol. 12821. Springer International Publishing.

https://doi.org/10.1007/978-3-030-86549-8_9.

Wang, Peng, Shuai Bai, Sinan Tan, et al. 2024. *Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution*. arXiv:2409.12191. arXiv. <https://doi.org/10.48550/arXiv.2409.12191>.